Searching and Algorithm Analysis

James Anderson

CS 3100/5100

Wright State University

Problems

- Problem – a mapping of *input* to *output*
- Algorithm – a method or a process followed to solve a problem
- A problem can have many algorithms

Example: Search

23	8	108	4	16	42	15

- Linear Search
  - Input: An array A of integers and a value v
  - Output:
    - True if v is an element of A
    - False if v is not in A

- How many elements checked if v is not in A?

Example: Search (again)

- Binary Search
  - Linear search worst case checks all elements
  - Sorted list

4	8	15	16	23	42	108

Constant Time Operations

- For a given processor, always runs in the same amount of time regardless of *n*

Constant

```
int x = 10;
int y = 20;

int z = x * y
```

Non-constant

```
for (int i = 0; i < n; i++) {
        sum += y;
}

String str3 = str1 + str2;
```

Operation	Example

Addition, subtraction, multiplication, and division of fixed size integer or floating point values.	`double w = 10.4;`
	`double x = 3.4;`
	`double y = 2.0;`
	`double z = (w - x) / y;`

Assignment of a reference, pointer, or other fixed size data value.	`int x = 1000;`
	`int* y = &x;`
	`bool a = true;`
	`bool& b = a;`

Comparison of two fixed size data values.	`int a = 100;`
	`int b = 200;`
	`if (b > a) {`
	`  ...`
	`}`

Read or write an array element at a particular index.	`int x = arr[index];` `arr[index + 1] = x + 1;`

Non-constant Running Time

```
sum = 0;
for (i = 0; i < n; i++) {
    sum++;
}
```

Non-constant Running Time

```
sum = 0;
for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++) {
        sum++;
    }
}
```

Runtime Complexity Example

```
sum = 0;
for (i = 0; i < n; i++) {
    for (j = 0; j < i; j++) {
        sum++;
    }
}
```

Series Sums

$$1 + 2 + 3 + \cdots + n = \sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

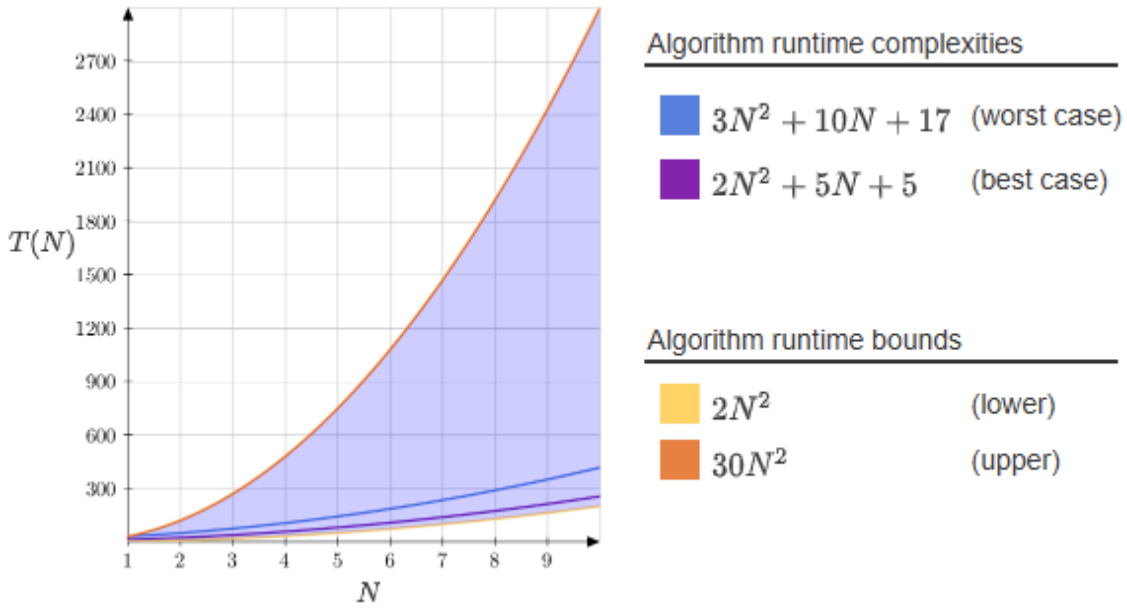
$$0 + 1 + 2 + 3 + \cdots + (n-1) = \sum_{i=0}^{n-1} i = \frac{n(n-1)}{2}$$

Runtime Complexity Example (2)

```
sum = 0;
for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++) {
        for (k = 0; k < n; k++){
            sum++;
        }
    }
}
```

Growth of Functions and Complexity

- Lower bound: A function $f(N)$ that is $\leq$ the best case $T(N)$, for all values $N \geq N_0$
- Upper bound: A function $f(N)$ that is $\geq$ the worst case $T(N)$, for all values $N \geq N_0$
- Preferred bound:
  - Single term polynomial
  - Bounds $T(N)$ as tightly as possible



Growth rates and asymptotic notations

- $\boldsymbol{O}$ **notation (Big-O notation):** growth rate for algorithm *upper - bound*
- $\boldsymbol{\Omega}$ **notation (Big-** $\boldsymbol{\Omega}$ **notation):** growth rate for algorithm *lower-bound*
- $\boldsymbol{\Theta}$ **notation (Big-** $\boldsymbol{\Theta}$ **notation):** growth rate that is both u*pper-bound* and *lower-bound*

Notation	General form	Meaning
O	$T(N) = O(f(N))$	A positive constant c exists such that, for all N ≥ N_0, $T(N) \leq c * f(N)$.
Ω	$T(N) = \Omega(f(N))$	A positive constant c exists such that, for all N ≥ N_0, $T(N) \geq c * f(N)$.
Θ	$T(N) = \Theta(f(N))$	$T(N) = O(f(N))$ and $T(N) = \Omega(f(N))$.

Big-O Notation

- How function behaves in relation to input size
- All functions with same growth rate are considered equivalent in Big-O notation
- $T(N) = O(f(N))$ is saying that $\mathrm{T}(N)$ is in Big-O of $f(N)$
- Big- O is set membership, should use $\mathrm{T}(N) \in O(f(N))$ but we just use $=$, either is fine
- Rules:
  - If $\mathrm{T}(N)$ is sum of several terms, keep highest ordered and discard the rest
  - If $\mathrm{T}(N)$ has a term that is a product of several factors, all constants (not in terms of N) are omitted

Runtime Growth Rate

Function	N = 10	N = 50	N = 100	N = 1,000	N = 10,000	N = 100,000
$log_2\ N$	3.3 µs	5.65 µs	6.6 µs	9.9 µs	13.3 µs	16.6 µs
N	10 µs	50 µs	100 µs	1,000 µs	10 ms	100 ms
$N\ log_2\ N$	0.03 ms	0.28 ms	0.66 ms	0.0099 s	0.132 s	1.66 s
N^2	0.1 ms	2.5 ms	10 ms	1 s	100 s	2.7 hours
N^3	1 ms	0.125 s	1 s	16.7 min	11.57 days	31.7 years
2^N	0.001 s	35.7 years	> 1,000 years			

zyBooks complexity graphing tool

Big-O Hierarchy

O	**Asymptotic Upper Bound**
O(1)	Constant
O($\log_a(n)$)	Logarithmic
O(n)	Linear
O($n \log_a(n)$)	$n \log n$
O(n^2)	Quadratic
O(n^3)	Cubic
O(n^r)	Polynomial ($r \geq 0$)
O(b^n)	Exponential ($b > 1$)
O($n!$)	Factorial (and exponential)

Function Comparison

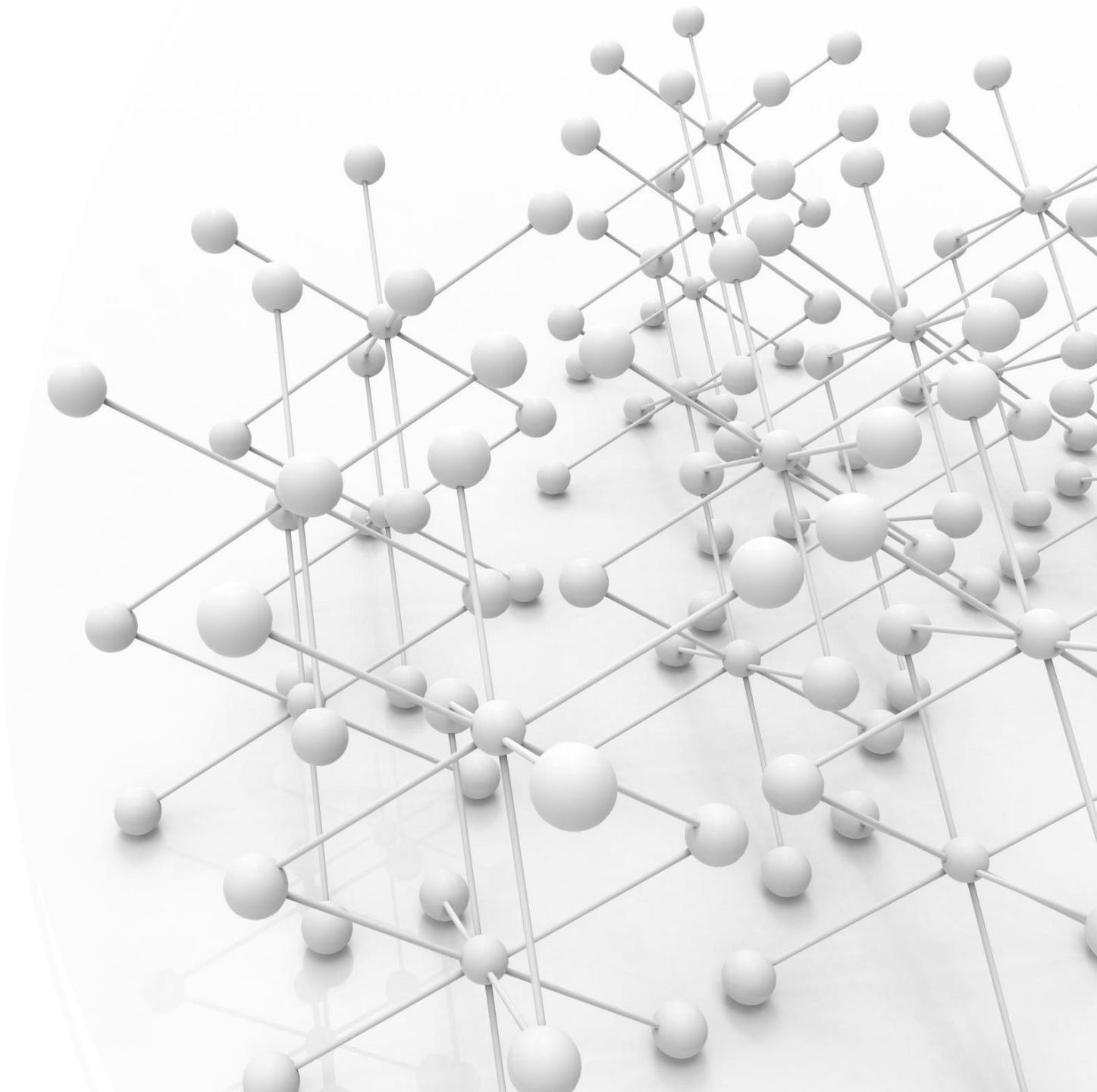
$T(N)$	O Ω Θ	$f(n)$
$2n + 13$		n^2
$n^2 + 3$		$7n^2$
n^3		$8n^3 + 4n^2 - n - 2$
$90 \log_2 n + 23$		$10{,}000{,}000$
2^n		n^2

Worst-, Best-, and Average-case

- Worst: Runtime complexity for input resulting in longest execution
- Best : Runtime complexity for input resulting in shortest execution
- Average: requires knowledge of statistical properties of input
  - E.g. Search: best case $O(1)$, worst case $O(N)$, average case:
  - If input space is all possible 32-bit integers, how likely will search key be first?

Recursive Algorithms

- Break problem into smaller subproblems
- Apply algorithm itself to solve smaller subproblems
- Base case: no more subproblems



Fibonacci sequence

- $Fib(0) = 0$
- $Fib(1) = 1$
- $Fib(n) = Fib(n - 1) + Fib(n - 2)$

```
int FibonacciNumber(int termIndex) {
   if (termIndex == 0) {
      return 0;
   }
   else if (termIndex == 1) {
      return 1;
   }
   else {
      return FibonacciNumber(termIndex - 1) + FibonacciNumber(termIndex - 2);
   }
}
```

Recursive functions

```
int Factorial(int N) {
   if (N <= 1) {
      return N;
   }
   else {
      return N * Factorial(N - 1);
   }
}


int CumulativeSum(int N) {
   if (N == 0) {
      return 0;
   }
   else {
      return N + CumulativeSum(N - 1);
   }
}


void ReverseArray(int* array, int startIndex, int endIndex) {
   if (startIndex >= endIndex) {
      return;
   }
   else {
      std::swap(array[startIndex], array[endIndex]);
      ReverseArray(array, startIndex + 1, endIndex - 1);
   }
}
```

Recursive Binary Search

- Compare key to middle element
- If match, return key's index
- Recursively search one half until found or empty

```
int BinarySearch(const int* numbers, int low, int high, int key) {
   if (low > high) {
      return -1;
   }

   int mid = (low + high) / 2;
   if (numbers[mid] < key) {
      return BinarySearch(numbers, mid + 1, high, key);
   }
   else if (numbers[mid] > key) {
      return BinarySearch(numbers, low, mid - 1, key);
   }
   return mid;
}
```

Analyzing Recursive Algorithms

- Recurrence relation: function on both sides of equation
- Big-O requires solving recurrence relation
- Simple: count number of function calls for N

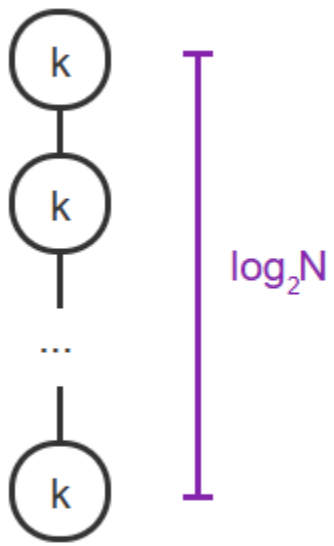
Input size	Number of function calls
1	1
2	2
4	3
8	4
16	5
32	6
...	...
N	$\log_2 N + 1$

If: $T(N) = O(1) + T(N / 2)$

Then: $T(N) = O(\log N)$

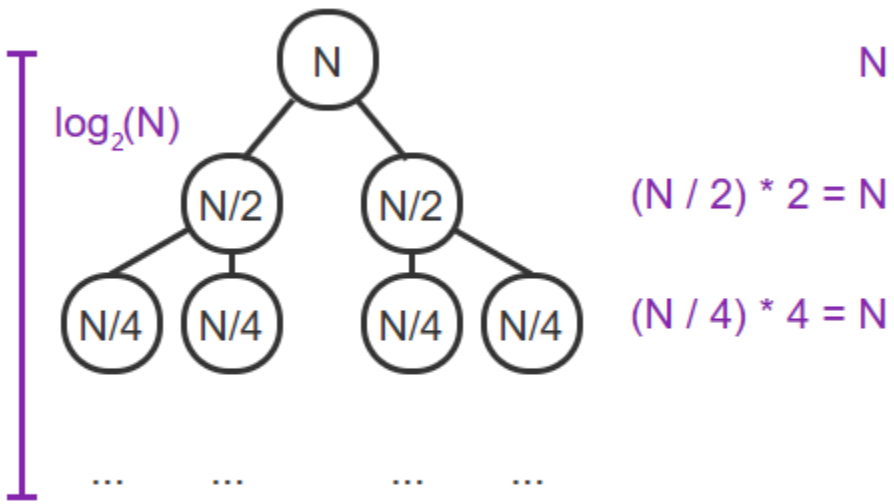
Recursion Tree

$T(N) = k + T(N / 2)$



$T(N) = O(\log N)$

$T(N) = N + T(N / 2) + T(N / 2)$



$T(N) = O(N \log N)$